

Buổi 13 — Feature engineering và chống rò rỉ

1. Mục tiêu

Sau buổi này bạn:

- Xây bộ **41 feature** mà **mọi giá trị đều có sẵn tại thời điểm ra dự báo**, và khai được mỗi feature "biết trước bao lâu".
- Viết đúng **lag** và **rolling**: vì sao phải `shift` trước khi `rolling`, và vì sao lag nhỏ nhất phải $\geq$ tầm dự báo.
- Dùng **Fourier term** thay cho hàng trăm biến giả mùa vụ, và mã hoá tuần hoàn sin/cos cho thứ/tháng.
- Tính **lịch âm Việt Nam** bằng code của chính mình (đúng cho mọi năm 2000–2035), tạo feature Tết và Giỗ Tổ.
- Tự viết `kiem_ro_ri`: cắt tương lai rồi tính lại — và biết vì sao **mốc cắt phải chọn có chủ đích**.
- Đo **cái giá của rò rỉ** bằng tiền thật, và hiểu vì sao cái giá đó lúc lớn lúc nhỏ.

2. Nhắc lại buổi trước

- **Nhật ký sự kiện** (buổi 11): Tết là sự kiện thật, không phải ngoại lai; không ngưỡng thống kê nào phân biệt được. Hôm nay ta biến nhật ký đó thành **feature**.
- **Bộ lọc nhân quả** (buổi 12): đầu ra tại t chỉ được phụ thuộc dữ liệu tới t . Hôm nay áp dụng nguyên tắc ấy cho mọi feature, và bài kiểm cũng cùng một ý tưởng: **đối/cắt dữ liệu tương lai, xem quá khứ có đổi không**.
- **Diễn dữ liệu là rò rỉ** (buổi 10): `interpolate(limit_direction="both")` dùng tương lai. Hôm nay nó xuất hiện lại như một trong bốn kiểu rò rỉ kinh điển.
- **Chấm trên chuỗi gốc** (buổi 12): mọi so sánh hôm nay đều trên cùng một tập kiểm, cùng một mục tiêu.

3. Trạng thái đầu buổi

Sau `cd lab && make up`:

Hạng mục	Trạng thái
Dữ liệu 1	<code>uci-online-retail-ii/</code> → 739 ngày doanh thu, 2009-12 → 2011-12, sha256 <code>572e36277c23</code>
Dữ liệu 2	<code>wikipedia-vi-tong/</code> — 3.653 ngày lượt xem vi.wikipedia (có 10 mùa Tết)
Dữ liệu 3	<code>eia930-balance-2024-h1/h2</code> — phụ tải ERCOT theo giờ kèm dự báo của chính nhà vận hành
Dữ liệu 4	<code>open-meteo-dallas-du-bao-luu-2024</code> — 8.784 giờ : nhiệt độ thật (ERA5) + bản dự báo đã lưu trước 1 và 3 ngày
Môi trường	Python 3.12.14; numpy 2.5.3, pandas 3.0.5, scikit-learn 1.9.1, holidays 0.104

Hạng mục	Trạng thái
code/ feature.py	âm lịch (Meeus), <code>feature_lich</code> , <code>feature_fourier</code> , <code>feature_tre</code> , <code>bo_feature</code> , <code>bang_biet_truoc</code> , <code>kiem_ro_ri</code> , <code>kiem_nhieu_muc_tieu</code> , <code>gia_cua_ro_ri</code> , <code>gia_tri_feature_tet</code>
Đang cố tình sai	4 feature rò rỉ (<code>tb_7</code> centered, <code>z_score</code> chuẩn hoá toàn bộ, <code>tb_theo_thu</code> target encoding toàn bộ, <code>y_dien_hai_chieu</code>); feature Tết hardcode ngày Tết 2011 ; <code>kiem_ro_ri</code> chỉ cắt một mốc và bỏ 10 dòng cuối
Triệu chứng	Bài kiểm rò rỉ báo "sạch" cho <code>tb_7</code> ; mô hình có MAE đẹp bất thường; feature Tết chỉ đúng cho năm 2011
<code>make_check</code> lúc này	ĐỎ: 9/13 test qua, 4 test hồng
Lưu ý	Buổi này tv không có ro_ri — bạn tự viết bài kiểm

4. Lý thuyết

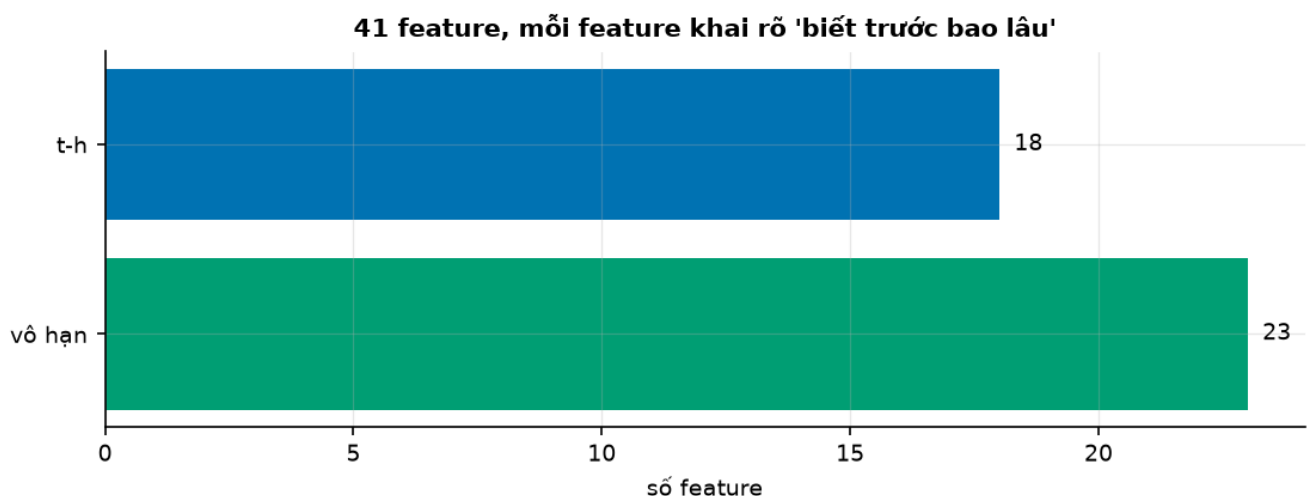
4.1 Nguyên tắc duy nhất

Feature cho y_{t+h} chỉ được dùng thông tin **có thật trong tay tại thời điểm t** .

Ba nhóm thông tin, ba mức "biết trước":

Nhóm	Ví dụ	Biết trước bao lâu
Lịch	thứ, tháng, Tết, Fourier	vô hạn — không bao giờ rò rỉ
Kế hoạch / dự báo	khuyến mãi đã lên lịch, dự báo thời tiết, dự báo phụ tải của nhà vận hành	tới thời điểm công bố
Quá khứ của chính chuỗi	lag, rolling	chỉ tới $t - h$

Mọi thứ khác — nhiệt độ **thật** của ngày mai, doanh số thật của tuần sau, "trung bình toàn bộ dữ liệu" — **không** có trong tay.



Đọc hình. Bộ feature của buổi: **41 cột**, trong đó 23 cột thuộc nhóm lịch (biết trước vô hạn) và 18 cột lấy từ quá khứ của chuỗi. Bảng này phải nộp **kèm mô hình** — nó là thứ giúp người review phát hiện rõ rĩ mà không cần đọc code.

4.2 Lịch âm Việt Nam — tự tính, đúng cho mọi năm

Tết theo **lịch âm**: ngày dương xê dịch 22–47 (buổi 11 đã đo). Một biến giả `is_tet` hardcoded một năm là vô dụng ngay năm sau.

Thuật toán (Meeus, *Astronomical Algorithms*): tính thời điểm **sóc** (trăng mới) và **kinh độ mặt trời**, quy về múi giờ địa phương, rồi áp quy tắc "tháng 11 âm là tháng chứa đông chí". Điểm mấu chốt: **múi giờ là tham số**.

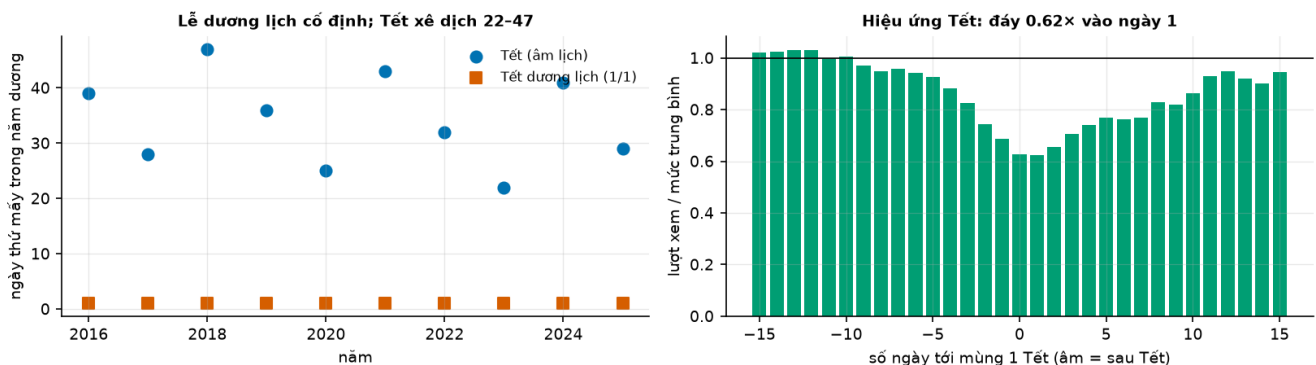
```
tet(2007, mui_gio=7) # 2007-02-17 (Việt Nam, kinh tuyến 105°Đ)
tet(2007, mui_gio=8) # 2007-02-18 (Trung Quốc, kinh tuyến 120°Đ)
```

Trong 2000–2035 có đúng **hai năm** lệch: **2007** và **2030**. Đây là lý do không dùng được thư viện lịch Trung Quốc cho dữ liệu Việt Nam. Bản tự viết đã đối chiếu **khớp 100% với thư viện holidays 0.104 cho cả 36 năm**, và Giỗ Tổ 2024 (10/3 âm) ra đúng 18/04/2024.

Từ đó sinh feature:

```
f["so_ngay_toi_tet"] = so_ngay_toi_tet(moc) # âm = đã qua Tết
f["truoc_tet_7"] = ((d > 0) & (d <= 7)).astype(int) # cửa sổ ảnh hưởng
f["trong_tet_7"] = (d.abs() <= 3).astype(int)
f["sau_tet_7"] = ((d < 0) & (d >= -7)).astype(int)
```

Vì sao cần feature lịch âm: hiệu ứng bám theo ngày ÂM, còn mọi feature dương lịch thì bám theo ngày dương



Đọc hình. Trái: Tết dương lịch luôn ở ngày 1, Tết âm lịch nhảy trong khoảng ngày 22–47. Phải: lượt xem vi.wikipedia theo **số ngày tới Tết** — đáy rơi đúng mừng 1 với **0,62x** mức trung bình. Hiệu ứng rất mạnh, nhưng chỉ nhìn thấy khi trục hoành là ngày **âm lịch**.

Giá trị đo được của nhóm feature này (dự báo lượt xem 1 ngày):

Bộ feature	MAE cả năm	MAE quanh Tết (± 10 ngày)
chỉ lag + lịch dương (36 feature)	132.137	167.769
+ feature Tết âm lịch (41 feature)	129.680	139.478 (−16,9%)

Cả năm chỉ cải thiện 1,9% — vì Tết chỉ chiếm ~6% số ngày. **Luôn báo cáo cải thiện ở vùng mà feature đó có tác dụng**, nếu không một feature quan trọng sẽ bị loại oan.

4.3 Feature lịch và Fourier

Mã hoá tuần hoàn: thứ 7 và chủ nhật phải "gần nhau", tháng 12 và tháng 1 cũng vậy. Số nguyên 0-6 không thể hiện điều đó:

$$\sin_k = \sin\left(\frac{2\pi k}{K}\right), \quad \cos_k = \cos\left(\frac{2\pi k}{K}\right)$$

Luôn đi thành cặp — chỉ dùng sin thì hai thời điểm khác nhau có cùng giá trị.

Fourier term cho mùa vụ dài: mùa vụ năm trên dữ liệu ngày cần 365 biến giả; thay bằng K cặp sin/cos với chu kỳ 365,25:

```
for i in range(1, K + 1):
    f[f"fourier_sin_{i}"] = np.sin(2 * np.pi * i * t / 365.25)
    f[f"fourier_cos_{i}"] = np.cos(2 * np.pi * i * t / 365.25)
```

$K = 3$ (6 cột) đủ cho hình dạng mùa vụ trơn; K lớn hơn bắt được đỉnh hẹp nhưng dễ overfit. Ưu điểm lớn: **số cột không phụ thuộc độ dài chu kỳ**, nên mùa vụ năm trên dữ liệu giờ (8.766 bước) vẫn chỉ tốn 2K cột.

4.4 Lag và rolling — đúng thứ tự

Hai quy tắc, học thuộc:

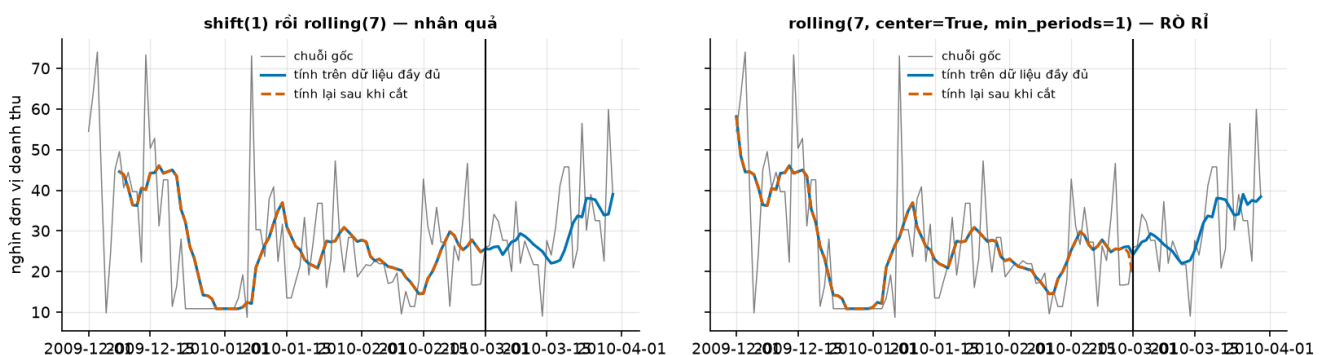
1. **Lag nhỏ nhất $\geq$ tầm dự báo h** . Dự báo 24 giờ tới mà dùng `lag_1` là dùng dữ liệu chưa có.
2. **shift(h) TRƯỚC, rolling SAU.**

```
tre = y.shift(h)                # đúng
f["tb_7"] = tre.rolling(7).mean()

f["tb_7"] = y.rolling(7).mean() # SAI: cửa sổ kết thúc tại t, gồm cả y_t
f["tb_7"] = y.rolling(7, center=True).mean() # SAI NẶNG: nửa cửa sổ nằm ở tương lai
```

`y.rolling(7).mean()` tại t gồm y_t — với $h = 1$ thì đó đã là rò rỉ. Còn `center=True` lấy 3 điểm **sau** t .

Bài kiểm tra: cắt dữ liệu tại vạch đen rồi tính lại. Trái: hai đường trùng khít. Phải: lệch tới 5 nghìn ngay TRƯỚC vạch



Đọc hình. Cùng một cửa sổ 7 ngày, cắt dữ liệu tại vạch đen rồi tính lại. Trái (`shift(1).rolling(7)`): đường "đầy đủ" và đường "tính lại" **trùng khít**, lệch 0,0. Phải (`rolling(7, center=True, min_periods=1)`): hai đường tách nhau ngay trước vạch, lệch tới **5.113** đơn vị. Chính chỗ tách ra đó là rò rỉ.

4.5 Bốn kiểu rò rỉ kinh điển

Kaufman et al. (2012) định nghĩa rò rỉ là "no-time-machine requirement" bị vi phạm; Kapoor & Narayanan (2023) cho thấy nó là nguyên nhân hàng đầu của khủng hoảng tái lập trong khoa học dùng ML. Bốn kiểu hay gặp nhất trong chuỗi thời gian:

Kiểu	Ví dụ	Vì sao rò rỉ
Rolling không shift	<code>y.rolling(7, center=True).mean()</code>	cửa sổ chứa y_t hoặc điểm sau t
Chuẩn hoá toàn bộ	<code>(y - y.mean()) / y.std()</code> trên cả chuỗi	trung bình/độ lệch chứa thông tin tập kiểm
Target encoding toàn bộ	doanh thu trung bình theo thứ, tính trên cả chuỗi	mỗi dòng "biết" trung bình của tương lai
Điền hai chiều	<code>interpolate(limit_direction="both"), bfill</code>	giá trị điền lấy từ tương lai

Thêm một kiểu đặc thù dự báo: **dùng giá trị thật của biến ngoại sinh thay cho bản dự báo** — mục 4.6.

4.6 Bài kiểm rò rỉ tự động

Ý tưởng giống buổi 12: **cắt tương lai, tính lại, so quá khứ**.

```
day_du = ham_feature(y)
for moc in cac_moc:
    cat = ham_feature(y[y.index <= moc])
    chung = day_du.index[day_du.index <= moc]
    # so TỪNG cột trên MỖI dòng ≤ moc; NaN vs số cũng là khác nhau
```

Bốn chi tiết quyết định bài kiểm có tác dụng hay không:

1. **So mọi dòng ≤ mốc cắt**, không bỏ "vài dòng cuối cho chắc" — đúng vùng đó mới lộ vi phạm.
2. **NaN so với số phải tính là khác nhau**. `rolling(center=True)` không có `min_periods` trả NaN ở mép; nếu bỏ qua NaN thì bài kiểm ra "sạch" trong khi feature đang rò rỉ.
3. **Chọn mốc cắt có chủ đích**. Rò rỉ kiểu `interpolate(limit_direction="both")` **chỉ lộ ra ở mép lỗ hổng**: nếu mốc cắt rơi vào chỗ dữ liệu đầy đủ thì hai lần tính giống hệt nhau. Bộ chấm của buổi này vì thế cắt đúng vào ngày bị khoét.
4. **Có bài kiểm thứ hai**. Cắt-tương-lai không bắt được `lag < tầm dự báo` (vì cắt ở đâu thì `lag_1` vẫn tính như nhau). Bài kiểm nhiều mục tiêu bổ sung: đổi y từ 70% chuỗi trở đi, feature của các dòng trước đó không được đổi.

Và bài kiểm thứ ba, rẻ nhất: **bảng "biết trước bao lâu"**. Nếu một cột không phân loại được vào ba nhóm ở mục 4.1 thì nó đáng ngờ.

Ba bài kiểm bắt ba nhóm lỗi khác nhau, không thay thế nhau được:

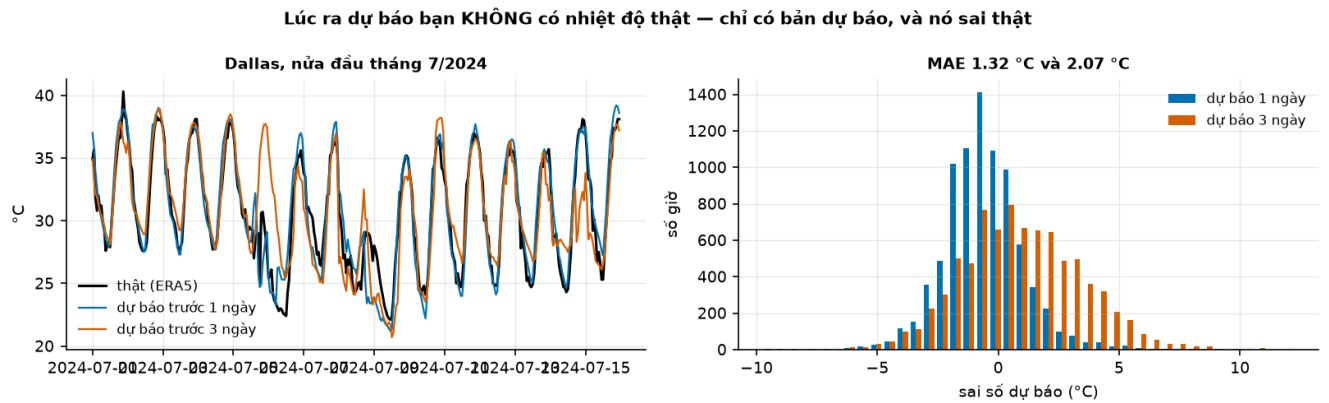
Bài kiểm	Bắt được	Bỏ sót
cắt tương lai	rolling không shift, chuẩn hoá/target encoding toàn bộ, điền hai chiều	<code>lag < tầm dự báo</code> ; dùng sai nguồn ngoại sinh
nhiều mục tiêu theo tầm h	<code>lag < h</code> , rolling có chứa y_t	feature ngoại sinh sai nguồn

Bài kiểm	Bắt được	Bỏ sót
bảng "biết trước"	dùng nhiệt độ thật, dùng dữ liệu đã hiệu chỉnh về sau	lỗi cài đặt bên trong một feature

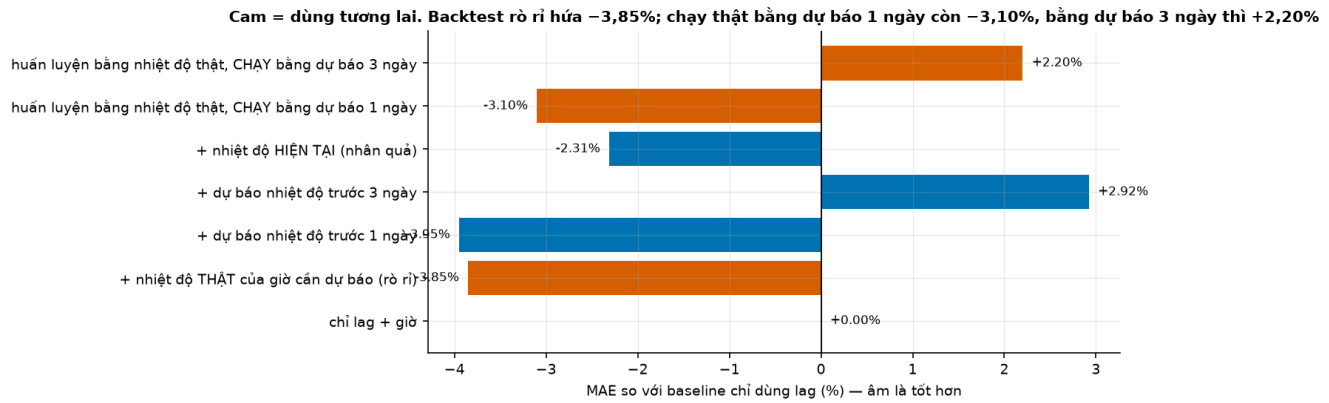
Ba bài đều rẻ (chạy trong vài giây) và nên nằm trong CI. Từ buổi này trở đi, mọi lab/cham/ của khoá đều có ít nhất một test loại này.

4.7 Cái giá của rò rỉ, đo bằng MW

Thí nghiệm: dự báo phụ tải ERCOT **24 giờ tới**. Nhiệt độ là biến giải thích quan trọng — nhưng lúc ra dự báo ta chỉ có **bản dự báo nhiệt độ**, không có nhiệt độ thật.



Độc hình. Trái: nửa đầu tháng 7/2024 ở Dallas — bản dự báo trước 1 ngày bám khá sát, bản trước 3 ngày lệch rõ. Phải: phân phối sai số, MAE **1,33 °C** (D+1) và **2,07 °C** (D+3) trên 8.292 giờ. Chính khoảng cách này quyết định cái giá của rò rỉ.



Độc hình. So với baseline chỉ dùng lag (MAE 1.909,68 MW):

Bộ feature	MAE (MW)	So với baseline
+ nhiệt độ THẬT của giờ cần dự báo (rò rỉ)	1.836,16	-3,85% ← con số backtest "hứa"
+ dự báo nhiệt độ trước 1 ngày	1.834,29	-3,95%
+ dự báo nhiệt độ trước 3 ngày	1.965,47	+2,92%
+ nhiệt độ hiện tại (nhân quả)	1.865,56	-2,31%
huấn luyện bằng thật, CHẠY bằng dự báo 1 ngày	1.850,50	-3,10%
huấn luyện bằng thật, CHẠY bằng dự báo 3 ngày	1.951,70	+2,20%

Ba bài học, và bài học thứ hai đi ngược trực giác:

1. **Backtest rò rỉ hứa nhiều hơn thực tế:** $-3,85\%$ hứa, $-3,10\%$ nhận được khi chạy bằng dữ báo D+1.
2. **Nhưng cái giá không phải lúc nào cũng lớn.** Dữ báo thời tiết D+1 tốt tới mức dùng nó gần bằng dùng sự thật (thậm chí hơi tốt hơn, vì mô hình được huấn luyện trên **cùng loại** đầu vào mà nó sẽ gặp lúc chạy). **Cái giá của rò rỉ = khoảng cách chất lượng giữa sự thật và thứ bạn thật sự có** — phải đo, đừng đoán.
3. **Với dữ báo D+3 thì từ $-3,85\%$ thành $+2,20\%$:** feature không những hết tác dụng mà còn làm hại. Cùng một feature, cùng một mô hình, khác nhau chỉ ở chất lượng nguồn thông tin.

Chú ý dòng cuối cùng còn một bài học nữa: nếu bạn **huấn luyện** bằng nhiệt độ thật rồi **chạy** bằng dữ báo, mô hình chưa từng thấy sai số của dữ báo nên tin nó quá mức. Muốn dùng biến ngoại sinh, hãy huấn luyện bằng **đúng loại dữ liệu sẽ có lúc chạy**.

4.8 Ghép dữ liệu ngoại sinh: `merge_asof` và cái bẫy mốc trùng

Biến ngoại sinh hầu như luôn đến ở tần suất khác và mốc lệch nhau. Ghép sai là một nguồn rò rỉ mà không bài kiểm nào bắt được, vì nó xảy ra **trước** khi feature được tính.

```
# ĐÚNG: mỗi dòng của trái chỉ được ghép với bản ghi phải có mốc ≤ mốc của nó
ghep = pd.merge_asof(trai.sort_index(), phai.sort_index(),
                    left_index=True, right_index=True,
                    direction="backward", tolerance=pd.Timedelta("3h"))
```

Ba chỗ hay sai:

1. **`direction="nearest"`** (mặc định của nhiều ví dụ trên mạng) lấy cả bản ghi **sau** mốc → rò rỉ.
2. **Không đặt `tolerance`**: khi nguồn phải bị thiếu một tuần, `merge_asof` vẫn kéo giá trị của tuần trước xuống và không báo gì.
3. **Mốc trùng ở bảng phải**: nếu bảng phải có hai bản ghi cùng mốc (bản công bố lần đầu và bản sửa lại), `merge_asof` lấy **bản cuối cùng theo thứ tự dòng** — tức có thể lấy đúng bản sửa về sau. Phải khử trùng **trước**, và khử theo quy tắc point-in-time ở mục 4.9.

Kiểm nhanh sau mỗi lần ghép: `ghep.index.is_monotonic_increasing`, số dòng không đổi, và tỷ lệ NaN của cột vừa ghép (nếu 0% một cách đáng ngờ thì nhiều khả năng `tolerance` chưa đặt).

Thêm một lưu ý múi giờ (buổi 10 đã gặp): EIA-930 ghi `UTC Time at End of Hour` — **mốc cuối giờ**, còn Open-Meteo ghi **mốc đầu giờ**. Lệch một giờ ở đây đủ để một feature "nhiệt độ cùng giờ" trở thành "nhiệt độ giờ sau", tức rò rỉ một bước.

4.9 Point-in-time: dữ liệu bị sửa về sau

Nhiều chuỗi kinh tế được **sửa lại** sau khi công bố (GDP, doanh số bán lẻ, số việc làm). Backtest dùng bản *hiện tại* là rò rỉ: năm 2019 bạn không có con số đã được sửa năm 2021.

Ba cách xử lý, theo thứ tự ưu tiên: (a) dùng **vintage/ALFRED-style** dữ liệu theo phiên bản; (b) nếu không có, **lùi mốc dùng dữ liệu** đúng bằng độ trễ công bố (ví dụ số liệu tháng 1 chỉ dùng từ 15/2); (c) tối thiểu, **ghi vào báo cáo** rằng backtest lạc quan hơn thực tế.

Dữ liệu buổi này có sẵn ví dụ tốt: EIA-930 có cột `Demand (MW)` và `Demand (MW) (Imputed)` — bản thô và bản đã hiệu chỉnh về sau. Dùng bản đã hiệu chỉnh để backtest là một dạng point-in-time sai.

5. Lab từng bước

Bước 1 — Xây bộ feature

```
cd lab && make up
env -u VIRTUAL_ENV uv run --no-sync --project 00-nen python ../code/feature.py
make check # 4/13 đỏ
```

Chạy `bo_feature(doc_ban_le().ffill())` và `bang_biet_truoc(f)`. Có cột nào không phân loại được không?

Gợi ý đọc bảng: nhóm "vô hạn" nên chiếm phần lớn (ở đáp án là 23/41). Nếu bộ feature của bạn gần như toàn nhóm "t-h" thì mô hình chỉ đang ngoại suy quá khứ và sẽ không biết Tết, lễ hay cuối tháng là gì.

Bước 2 — Tự viết `kiem_ro_ri`

Sửa `kiem_ro_ri` để: cắt tại **nhiều** mốc, so **mọi** dòng $\leq$ mốc, coi NaN vs số là khác nhau. Kiểm bằng 4 hàm feature rò rỉ mà bộ chấm cung cấp — phải bắt đủ 4 và **không** gán cờ 2 feature nhân quả.

Bước 3 — Sửa 4 feature rò rỉ

Dùng chính bài kiểm vừa viết để tìm và sửa. Sau đó chạy `kiem_nhieu_muc_tieu` — bài kiểm thứ hai.

Bước 4 — Lịch âm và giá của rò rỉ

Sửa feature Tết để đúng cho **mọi** năm (không hardcoded). Kiểm: `tet(n)` khớp `holidays` cho 2000–2035, và `tet(2007, 7) != tet(2007, 8)`. Cuối cùng chạy `gia_cua_ro_ri()` và viết ba câu kết luận cho sếp.

```
make check # 13/13 xanh
```

6. Lỗi thường gặp & cách chẩn đoán

Triệu chứng	Nguyên nhân	Chẩn đoán	Sửa
Backtest đẹp, sản xuất tệ	rò rỉ	<code>kiem_ro_ri</code> nhiều mốc	sửa feature, chạy lại backtest
Bài kiểm rò rỉ báo "sạch" mà vẫn nghi	bỏ dòng cuối / bỏ qua NaN / mốc cắt sai chỗ	in số dòng được so	so mọi dòng $\leq$ mốc, <code>equal_nan=True</code> , cắt đúng vào lỗ hổng
<code>rolling</code> bắt được cả hiện tại	quên <code>shift(h)</code>	xem cửa sổ có chứa y_t không	<code>y.shift(h).rolling(w)</code>
Mô hình 24 giờ dùng <code>lag_1</code>	lag < tầm dự báo	<code>kiem_nhieu_muc_tieu</code>	lag nhỏ nhất $\geq h$
Scaler "biết" tập kiểm	<code>fit</code> trên toàn bộ	so <code>scaler.mean_</code> với <code>X_train.mean()</code>	<code>fit</code> chỉ trên train
Target encoding quá tốt	tính trên cả chuỗi	<code>kiem_ro_ri</code>	tính expanding có shift, chỉ dùng quá khứ

Triệu chứng	Nguyên nhân	Chẩn đoán	Sửa
Feature Tết chỉ đúng một năm	hardcode ngày	kiểm <code>so_ngay_toi_tet</code> ở mọi năm	tính từ lịch âm
Ngày Tết lệch một ngày	dùng thư viện lịch Trung Quốc (UTC+8)	so <code>tet(n, 7)</code> và <code>tet(n, 8)</code>	tính ở kinh tuyến 105°Đ
Feature "cải thiện 1%" bị loại oan	báo cáo trung bình cả năm	tách vùng có tác dụng	báo MAE quanh Tết riêng
Mô hình tin dự báo thời tiết quá mức	huấn luyện bằng giá trị thật	so MAE khi chạy bằng dự báo	huấn luyện bằng đúng loại dữ liệu sẽ có
Mùa vụ năm ngốn 365 cột	dùng biến giả	đếm số cột	Fourier với K nhỏ
Thứ 7 và chủ nhật "xa nhau"	mã hoá số nguyên	vẽ sin/cos	dùng cặp sin và cos

7. Bài tập về nhà

- Quét K của Fourier.** Với $K \in \{1, 2, 3, 5, 10\}$, đo MAE trên tập kiểm của chuỗi lượt xem. K nào tốt nhất, và bạn giải thích thế nào khi K lớn làm sai số tăng?
- Rò rỉ tinh vi.** Thêm feature "doanh thu trung bình 7 ngày của **cùng mã hàng** trong toàn bộ dữ liệu" rồi chạy `kiem_ro_ri`. Nó có bị bắt không? Nếu không, vì sao — và bài kiểm nào bắt được?
- Point-in-time.** Với EIA-930, so backtest dùng `Demand (MW)` (bản thô) và `Demand (MW) (Imputed)` (bản hiệu chỉnh về sau). Chênh lệch MAE là bao nhiêu, và bạn báo cáo con số nào?

8. Tiêu chí "Xong khi"

- [] `make check` xanh 13/13.
- [] Bộ feature ≥ 40 cột, **qua bài kiểm rò rỉ 100%** (cả bài của bạn lẫn bài độc lập của bộ chấm).
- [] `kiem_ro_ri` của bạn bắt đủ 4 kiểu rò rỉ kinh điển và không báo động giả.
- [] Feature Tết chạy đúng cho **mọi năm 2000-2035** (đối chiếu `holidays`), và giải thích được vì sao 2007 và 2030 khác Trung Quốc.
- [] Nộp bảng "biết trước bao lâu" cho từng feature — không cột nào rơi vào nhóm "KHÔNG BIẾT TRƯỚC".
- [] Nộp bảng giá của rò rỉ, và trả lời: backtest hứa bao nhiêu, chạy thật được bao nhiêu, vì sao chênh.

9. Đọc thêm

- Kaufman, S., Rosset, S., Perlich, C. & Stitelman, O. (2012). Leakage in Data Mining. *ACM TKDD* 6(4).
- Kapoor, S. & Narayanan, A. (2023). Leakage and the reproducibility crisis in machine-learning-based science. *Patterns* 4(9).
- Hyndman & Athanasopoulos, *FPP3* ch. 7.4 "Useful predictors" — <https://otexts.com/fpp3/useful-predictors.html>
- Meeus, J. (1998). *Astronomical Algorithms*, 2nd ed. — ch. 25 (mặt trời), ch. 49 (pha mặt trăng).
- Hồ Ngọc Đức — Âm lịch Việt Nam: <http://www.informatik.uni-leipzig.de/~duc/amlich>
- `holidays` — <https://github.com/vacanza/holidays> (lễ Việt Nam, có nghỉ bù)
- Open-Meteo Previous Runs API — <https://open-meteo.com/en/docs/previous-runs-api>

- EIA-930 Hourly Electric Grid Monitor — <https://www.eia.gov/electricity/gridmonitor/>
- pandas windowing — https://pandas.pydata.org/docs/user_guide/window.html